

note:

By default the `writeln` func will not add a new line.

A new line should be added manually.

4) Appending:

Writing data into a file without erasing the existing data (appending)

open the file in append mode.

The same func for writing, `write` & `writeln`, can be used for appending.

5) Closing:

The `close` func is used to close a file.

note:

When the file is opened using `with block` it automatically closes the file.

⊗ closing a file is imminent for the changes to appear in the file.

File pointers

→ `tell()` - tells the location of the cursor.

→ `seek()` - used to the cursor position.

count of words in a file:

1. Open the file in read

2. read the file and store

its content (use `read()`)

3. Use the `split` func on the content (`content.split()`)

4. This gives you a list

of words

5. Return the length of

list

Performing multiple operation on file parallelly

mode	operation
r+	read and write
w+	overwrite and read
a+	append and read

Reading a log file:

log file:

A file that contains logs (update about a working application)

Levels:

1. **INFO** - gives the information about the current state of application.
2. **Debug** - gives information on the process with a debugging overview
3. **ERROR** - gives application errors

```
f = open("applog.log", "r")
```

```
logs = f.readlines()
```

```
infos = []
```

```
debugs = []
```

```
errors = []
```

```
for log in logs:
```

```
    if ("INFO" in log):
```

```
        infos.append(log)
```

```
    elif ("DEBUG" in log):
```

```
        debugs.append(log)
```

```
    elif ("ERROR" in log):
```

```
        errors.append(log)
```

```
print(log summary)
```

```
print(total number of INFO logs, len(infos))
```

```
print(len(infos))
```

```
print(total no. of DEBUG logs, len(debugs))
```

```
print(len(debugs))
```

```
print(total no. of ERROR logs,
```

```
len(errors))
```

O/P

log summary

Total number of INFO logs: 42

Total no. of DEBUG logs: 20

Total no. of ERROR logs: 6

using `r+` mode

It allows us to read from a file and write into a file as well. Starts writing from the cursor position writing process.

It does partial overwriting (i.e.) It overwrites only the char equal to the length of char being written and not the entire file.

using `w+` mode

It allows us to perform read and write operation on a file writing process.

It does complete overwriting when a file is opened in `w+` mode, the entire content of the file is overwritten.

using `a+` mode

Allows us to read or append to a file writing process.

It does no overwriting (i.e.) the previous content is erased.

comma separated (CSV files).

A CSV file is a file that contains a no. of lines where each line represents 1 row in table.

Each line has no. of values where the values are separated by a delimiter (usually a comma).

The first line represents the header row in the table.

Reading CSV file:

1. Import the CSV module

`reader()` - The `reader()` reads a CSV file and returns a CSV reader obj which contains a num of lists where each list represents a line in the CSV file / a row in table

```
import os
```

```
os.chdir("D:\\batch 21\\")
```

```
import csv
```

```
f = open('data.csv', 'r')
```

```
rows = csv.reader(f)
```

```
for row in rows:
```

```
    print(row)
```

```
dp ['name', 'age', 'city']  
dp ['John', '25', 'New York']  
E
```

2. `DictReader()`

It contains a no. of dictionaries where each dictionary represents a line in the CSV file or a row in a table in the form where the values are presented as key value pair in

which the keys are attributes of the 1st line in the CSV file or header row in the table.

```
f.seek(0)
```

```
rows = csv.DictReader(f)
```

```
for row in rows:
```

```
    print(row)
```

```
dp
```

```
{ 'name': 'John', 'age': '25',  
  'city': 'New York' }
```

common errors in file handling:

1. File Not Found Error

This error is thrown when you try to read a file that does not exist.

note:

creating a file

open the file in write mode, if file exist already it overwrites the file. if file doesn't exist

1. create a new file

2. Permission Error

This error is thrown when you try to perform some action beyond your access in a file.

3. Is A Directory Error

Occurs when you perform file operation on a directory.

4. Value Error:

Occurs when you try to perform input output operation on a closed file

5. Not A Directory Error:

Occurs when you try to perform directory operation on a file.

Types of statement in a program include

1) Normal statement
Eg: $a \times b$

2) critical statement -

These statements may throw an exception

Eg: $a // b$

try block:

All critical statements are kept inside try block

except block:

contains code to handle exceptions

finally block:

The code inside finally block is executed irrespective of an exception being thrown or not

Eg:

try:

except ValueError:

print("can't perform
input out I/O operation on a
closed file")

except exception:

writing into csv file:

* import csv module

* writer() - It is used to
write into a csv file
and returns a csv
writer object.

* The writerow() is
used to write a row
into the table

* writerows() - It is
used to write multiple
rows into a table
It takes a list containing
multiple list where each
list represents one row
from the table

note:

By default the ^{writer obj.} writerow() ~~inserts~~
inserts a new line after
writing each row, to
avoid this pause a 3rd
argument newline whose
value = an empty string.

ⁱⁿ the open function,

f = open('data.csv', 'a', newline='')

except FileNotFoundError:

f = open("data.txt", "w")

f.close()

except PermissionError:

print("Permission denied")

except IsADirectoryError:

print("You are trying to

perform file operation on file")

except NotADirectoryError:

print("You are trying to

perform folder operation on file")

```

writer = csv.writer(f)
writer.writerow(data)
f.close()

```

`dictwriter()` - used to write multiple data into the csv file using a list of dictionaries where each dictionary represents a row in a table / a line in csv file

The `dictwriter()` take 2 args: 1 - file pointer & 2 - header row (field)

```

write_header() -
f = open('mark.csv', 'w', newline='')
writer = csv.DictWriter(f, fieldnames=...)
writer.writerow(...)
f.close()

```

with `open('mark.csv', 'a', newline='')` as f:

```

writer = csv.DictWriter(f, fieldnames=...)
data = []
data.append({'Name': 'Ravi', 'Age': '010', 'Sub': 'am', 'Mark': '90'})
data.append({})
data.append({})
writer.writerow(data)

```

```

f = open('mark.csv', 'r')

```

working with directories

The `os` module is used to work with directories

It contains a num of func that let you perform imp operation on directories.

1) To get the current directory.

`os.getcwd()`

2) To change dir

`os.chdir()`

3) To get the contents of a dir.

`os.listdir()`

4) To create a new folder

`os.mkdir()` (Folder name)

5) To remove a dir

`os.rmdir()` (folder name)

⊗ It will only work for an empty dir

6) To delete a file

`os.remove()` (file name)

shutil module:

Import shutil as sh

1) copy:

sh.copy (source, destination)

Eg: sh.copy('data.csv', 'student')

data.csv is copied and pasted in student folder.

2) move:

sh.move(' ', ' ')

3) To remove a folder that has some files

sh.rmtree(folder name)

It deletes a folder even if it contains some files

Getting Individual columns from a csv file

* use the index of the constrained column

Eg:

data = reader
for row in data:
print(row[0])

Name
Aasija
Ameer
Mohithra

P.set(0)

CSV file reader:

+ open the file in read mode.

+ Read its content

+ using dictreader

+ Using a for loop

traverse each row dictionary given by the dictreader()

+ Apply filtering using

conditional statements

f = open('data.csv', 'r')

data = csv.DictReader(f)

for row in data:

if row['department']:

skip() =

else:

print(row)

o/p

pg Name: 'Aasija', city: 'Hosur'

Dept: 'CSF'

1 - - - - - print row

*Writing into a csv file.

+Working with modules:

A module is a python file that contains variables, functions even classes, which can be used by other programs to perform certain task without the need of writing code to perform that task.

Eg: The os module contains several variables and functions that helps us perform several operation on the operating system including creating a new directory, changing working director, deleting directory etc.

*Using external modules inside your programs

The import keyword is used to import an external module into your program.

Eg:

```
import os
import csv
```

Syntax:

```
import modulename
```

*Using Alias

Keyword - As

The as keyword is used to create an alias.

alias means also known as

```
import pandas as pd
```

from keyword:

The from keyword

used to call a specific

structure from a module

instead of importing

whole module

Syntax:

```
from modulename
```

```
structurename
```

Eg:

There is a sub module

in called Data Frame

which is present in

Pandas module

and used to work with data frames

Eg:
from Pandas import DataFrame

this statement will only import the DataFrame from the Pandas module

instead of importing the whole Pandas module.

*Advantage:

→ Reduces memory usage

*Disadvantage:

→ other structures cannot

be used

creating modules

A module is just a python file (.py file) which contains specific instructions

So to create your own custom module just create a python file.

*Module Search path:

When you import a module Python searches the module in a specific order. This order is called as MSP.

*Order of searching:

- 1) Current directory
- 2) Python environment variable
- 3) standard package libraries.

JSON:

→ JavaScript Obj Notation

JSON is a way of exchanging information b/w diff pages in a website.

In JSON objects are enclosed inside curly braces. Each obj has a set of attributes, where each attribute is present as key value pair.

Eg:

```
{  
  "Name": "Mohithrad",  
  "city": "Madurai",  
  "Dept": "ECE"  
}
```

type - object.

JSON

object

string

number

boolean

null

PYTHON

dict

str

int, float

boolean

none

To work with JSON data, the functions of the ~~JSON~~ ^{json} module can be used.

It provides the fn:

* load - convert JSON obj into python obj.

* loads - converts JSON string to python obj

* dump - converts python obj into JSON obj.

* dumps - convert python obj into json string

The process of converting json obj into python obj is called as parsing.

```
[  
  {  
    "Name": "Sai Sathya",  
    "city": "Arivayalur",  
    "dept": "ATPS",  
  },  
  {  
    "Name": "Aashja",  
    "city": "Hosur",  
    "dept": "CSE",  
  },  
]
```

```
>>> with open('data.json',  
              'r') as f:
```

```
    data = json.load(f)
```

```
    for line in data:  
        print(line)
```

```
or  
[  
  - - -  
]
```

```
[  
  - - -  
]
```

```
>>> data[0]  
{  
  - - -  
}
```

```
>>> type(data[0])
```

```
<class 'dict'>
```

```
js = {'Name': 'Vij', 'city': 'Chennai',  
      'dept': 'CSE'}
```

```
type(js)  
<class 'dict'>
```

```
py = json.loads(js)  
py
```

```
{ - - - }  
type(py)  
<class 'dict'>
```

dump:

used to convert py
obj into json obj.
process - serialization.

with open('data.json', 'w')
as f:

```
data = [  
    { "Name": "Vij",  
      - - -  
    },  
    { - - - },  
    { - - - },  
    { - - - }  
]
```

```
dump(data, f)
```

data json

```
[ { - - - }, { - - - } ]
```

dumps

```
>>> py = { - - - }
```

```
>>> type(py)
```

```
<class 'dict'>
```

```
>>> js = json.dumps(py)
```

```
>>> type(js)
```

```
<class 'str'>
```

```
>>> js
```

O/P:

```
{ - - - }
```

Useful Built-in Modules:

There are several
built-in modules that
have pre-defined code which
help us perform certain
operations without
writing our own logic.

* math module - It provides
several inbuilt fn to perform
mathematical operations.

1) Constants:

π - math.pi (3.14159...)

∞ - math.inf

$-\infty$ - -math.inf

2) Rounding values:

math.ceil (nearest greater)

Integer eg: 3.57 $\rightarrow$ 4

math.floor (nearest small)

Integer eg: 3.57 $\rightarrow$ 3

math.bunc (removes the decimal part)

3) log & sq-root

math.sqrt - gives the sq-root of a number (returns float type) eg: $\sqrt{9} \Rightarrow 3.0$

math.log10() - gives the log value of a number for the base of 10

To find the number of integer:

$$\text{int}(\text{math.log}_{10}(n)) + 1$$

+ count the num of digit in n

4) To find the absolute value in a number.

math.fabs - gives the abs value of a number (positive value)

5) probability fns

math.comb(n, r)

$$nCr = \frac{n!}{(n-r)! r!}$$

math.perm(n, r)

$$nPr = \frac{n!}{(n-r)!} \text{ or } nCr \times r!$$

math.factorial() - gives the factorial of a number

6) GCD & LCM:

math.gcd() - gives gcd of a number.

math.lcm() - gives lcm of a number.

Random Module:

* Randomization - it is the process of creating random values

* Random module provides fns that implement pseudo randomization. (will appear as usual randomization but follows an algorithmic approach to generate random values)

Imp fns:

1) ~~randint~~ random.randint(start, end)

start and end are inclusive

2. random.randrange(start, end)

start inclusive end exclusive

3. random.random() - gives

a random floating pt value b/w 0.0 & 1.0

both are exclusive

4. random.uniform(a, b)

gives a random floating point b/w a & b

a & b - exclusive.

5. random.choice(sequence)

6. random.shuffle - shuffles

a list of values.

Statistics module:

used to perform

several operations related to statistics.

this module provides

several fn to perform statistical operations

1. Statistics.mean

$$\bar{x} = \frac{\sum x}{n}$$

2. Statistics.median
used to get the median of a data set. A

particular value in data set, that is half way b/w the values.

eg: -7 1 3 -3 7

median: 1 bec

-7 -3 ① 3 7

3. Statistics.mode - gives the most frequent element in the data set.

7 5 5 2 5 9

ans: 5

4. Statistics.standard deviation

STDEV gives the SD

in the data set.

5. Statistics.variance

gives the avg diff b/w each value & the mean value

7	5
5	2
2	5
5	9

$$\frac{1}{n} \sum (x_i - \bar{x})^2$$

data time module:

This module provides
for that helps us work
with time & date

1. date time . date (year, mm, dd)

2. To get current date

date
datetime, today ()

3. To create time

datetime.time (hr, min, sec)

4. To get current time

datetime.
^ datetime.now()

5. datetime.strptime time

format specifiers:

% H	for hour
% M	Minutes
% S	seconds
% d	day
% m	month
% y	year

```
date.strftime("%d-%m-%y  
%H:%M:%S")
```

Package 3:

It is a collection of modules that contain a

— index — "Py file

import keyword is used to import a package from keyword is used to import a specific module from package as keyword is used to create a alias of package.

Installing packages:

The pip command is used to install external packages.

~~Insta~~
pip install packagename

installing a particular
version.

pip install packagename version

Install same video

2. Install same or smaller

1. X-Version 201

Install some or latest

updating

no install -- upgrade package

Uninstall

pip uninstall packagename.

Virtual Environment:

File Handling using python

File:

The file is used to store data for a long term. The data inside of a program is volatile (i.e.) it has no existence once the program closes.

Types of files:

1. Text files - stores data in the form of characters (e.g.) .text, .docx, .py

2. Binary files - stores binary data. Most multimedia data are stored as binary files (e.g.) .jpg, .mp3, .mp4

Need for files:

* Files allow us to store data for a long time (i.e.) data inside a file is non-volatile.

* Allows data transmission from one location to another location.

path

location of a file

→ absolute path:

→ complete path: from the root directory to the file location

→ relative path:

path of a file in relation to the current folder is called as relative path.

Absol eg:

C:\user\picture\series

rel eg:

Pictures/happy.jpg

Operations on a file:

→ create file

→ open the file

→ read/write to the file

→ close the file

opening the file:

The open function is used to open a file. It takes two args: file name and mode.

file name: Name of the file

mode: The type of operation perform

Mode	Operation
r	read
w	write
a	append

Syntax:

`f = open(filename, mode)`

The open function returns a file pointer

opening a file using with block

The with block automatically closes the file

`with open(file_name, mode) as f:`

2) Reading a file:

* Open the file in read mode

* There are 3 func to read from a file

* read: reads the entire content of file

* readline: Reads one line from the file

* readlines: Returns a list of strings where each string represents a line from the file

After reading the file close the file

3) Writing into a file:

* Open the file in write mode

* ~~(x)~~ If data is already present in the file, it is erased (overwrite)

* There are 2 func to write into a file

* write: writes the content into a file

* writelines: Returns a list of strings these strings are returned into the file